



Práctica 4 Parte 1

Vazquez Rangel Guillermo Larzen

2024171016

IDGS17

Gerardo Ramírez Villarreal

Seguridad en el Desarrollo de Aplicaciones

Objetivo

Extender la solución de logging implementada previamente en VulnerableApp (Serilog hacia consola, archivo y Seq) incorporando una plataforma de observabilidad basada en Grafana, Loki y Promtail, sin modificar la aplicación ni su lógica de negocio, de forma que los mismos registros puedan centralizarse y visualizarse desde una herramienta adicional.

docker-compose.yml

```
version: "3.9"

services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - observabilidad_net

  loki:
    image: grafana/loki:latest
    container_name: loki
    ports:
      - "3100:3100"
    volumes:
      - loki_data:/tmp/loki
    networks:
      - observabilidad_net

  promtail:
    image: grafana/promtail:latest
    container_name: promtail
    volumes:
      - ./Logs:/var/log/VulnerableApp:ro
      - ./promtail-config.yaml:/etc/promtail/config.yml:ro
    command: -config.file=/etc/promtail/config.yml
    networks:
      - observabilidad_net

networks:
  observabilidad_net:
    driver: bridge
```

```
volumes:
  grafana_data:
  loki_data:
```

promtail-config.yaml

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: vulnerable_app_logs
    static_configs:
      - targets:
          - localhost
        labels:
          job: vulnerableapp
          __path__: /var/log/VulnerableApp/*.txt
    pipeline_stages:
      - regex:
          expression: "(?P<categoria>(Security|Audit))"
      - labels:
          categoria:
```

Actividad de Análisis — Seq vs. Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda textual	Índice completo sobre el mensaje y las propiedades estructuradas; lenguaje de consulta propio tipo SQL.	LogQL: primero selecciona el stream por labels (índice), luego filtra el texto de esas líneas (=, ~, regex) sin indexar el contenido completo.
Dashboards	Paneles y señales guardadas, orientados a logs de una sola aplicación .NET.	Grafana permite dashboards altamente personalizables combinando múltiples fuentes (Loki, Prometheus, etc.) en un mismo panel.

Característica	Seq	Grafana + Loki
Alertas	Alertas basadas en consultas guardadas, con notificación por correo/webhook.	Grafana Alerting centraliza reglas de alerta sobre cualquier fuente conectada, con múltiples canales (email, Slack, PagerDuty, etc.).
Consultas	Filtra por propiedades estructuradas emitidas por Serilog (Level, CorrelationId, Message) con su propio lenguaje.	LogQL combina selección por labels + operadores de línea y funciones de agregación (rate(), count_over_time()).
Visualización	Interfaz especializada en logs estructurados de una aplicación .NET.	Vistas mixtas: series de tiempo, tablas y mapas de calor, combinando logs con métricas de otras fuentes.
Uso principal	Depuración y análisis detallado durante el desarrollo de una aplicación específica.	Observabilidad centralizada multi-servicio en un entorno de operación/producción.

¿Qué modificaciones realizó en Promtail?

Se integró un pipeline de procesamiento (pipeline_stage) en el job vulnerable_app_logs. Mediante una expresión regular (regex), el sistema analiza cada entrada de log para identificar las palabras clave 'Security' o 'Audit'. Una vez detectadas, la etapa de etiquetas (labels) promueve estos valores a metadatos del stream. Esto permite que cada línea sea clasificada automáticamente bajo una categoría específica sin necesidad de modificar el código fuente de VulnerableApp

¿Cómo utiliza Loki las labels?

Loki optimiza el rendimiento evitando la indexación completa del cuerpo del mensaje. El sistema utiliza las combinaciones de etiquetas (labels) como índice principal para organizar los datos. Cada combinación única, como {job="vulnerableapp", categoria="Security"}, define un stream independiente donde se almacenan los bloques de logs, garantizando una recuperación de datos eficiente.

¿Qué ventajas ofrecen para realizar consultas con LogQL?

Eficiencia en consultas: Reduce el volumen de datos a inspeccionar al filtrar por etiquetas antes de realizar la búsqueda de texto, incrementando significativamente la velocidad de respuesta.

Consultas complejas: Facilita la ejecución de búsquedas que combinan criterios estructurados con búsqueda de texto libre (p. ej. {categoria="Audit"} |= "fallido").

Analítica: Permite la creación de paneles de control y alertas basadas en categorías de negocio, facilitando la generación de métricas (como conteo de eventos por minuto) sin requerir el procesamiento de logs en tiempo real.

Pregunta de Reflexión

Incorporar una plataforma de observabilidad sin modificar la aplicación existente permite separar la responsabilidad de generar telemetría (que ya cumplía VulnerableApp mediante Serilog) de la responsabilidad de almacenarla, indexarla y visualizarla, la cual queda delegada por completo a la infraestructura (Promtail, Loki, Grafana). Esto reduce el riesgo de introducir errores en la lógica de negocio solo para 'agregar observabilidad', evita ciclos de despliegue de la aplicación cada vez que se quiere cambiar cómo se visualizan o correlacionan los logs, y permite versionar y evolucionar la plataforma de observabilidad (agregar nuevas fuentes, dashboards o alertas) de forma completamente independiente al código de la aplicación.

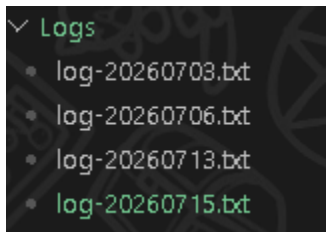
En un sistema en producción, esta arquitectura facilita la evolución porque la aplicación solo necesita mantener un contrato estable de logging (formato de mensajes, niveles, propiedades estructuradas como CorrelationId), mientras que la capa de observabilidad puede escalar horizontalmente, sumar nuevas fuentes (métricas con Prometheus, trazas con Tempo) o cambiar de proveedor sin tocar ni volver a desplegar los servicios de negocio, siguiendo el mismo principio de separación de responsabilidades que ya aplica el patrón de middleware usado para el CorrelationId.

Evidencias

1. VulnerableApp generando registros

```
Huesos@DESKTOP-HLGDDSV MINGW64 ~/Documents/VulnerableApp (logs_3)
$ dotnet run
Usando la configuración de inicio de C:\Users\Huesos\Documents\VulnerableApp\Properties\launchSettings.json...
Compilando...
[18:27:09 INF] User profile is available. Using 'C:\Users\Huesos\AppData\Local\ASP.NET\DataProtection-Keys' as key repository and Windows DPAPI to encrypt keys at rest.
[18:27:09 INF] Now listening on: http://localhost:5018
[18:27:09 INF] Application started. Press Ctrl+C to shut down.
[18:27:09 INF] Hosting environment: Development
[18:27:09 INF] Content root path: C:\Users\Huesos\Documents\VulnerableApp
```

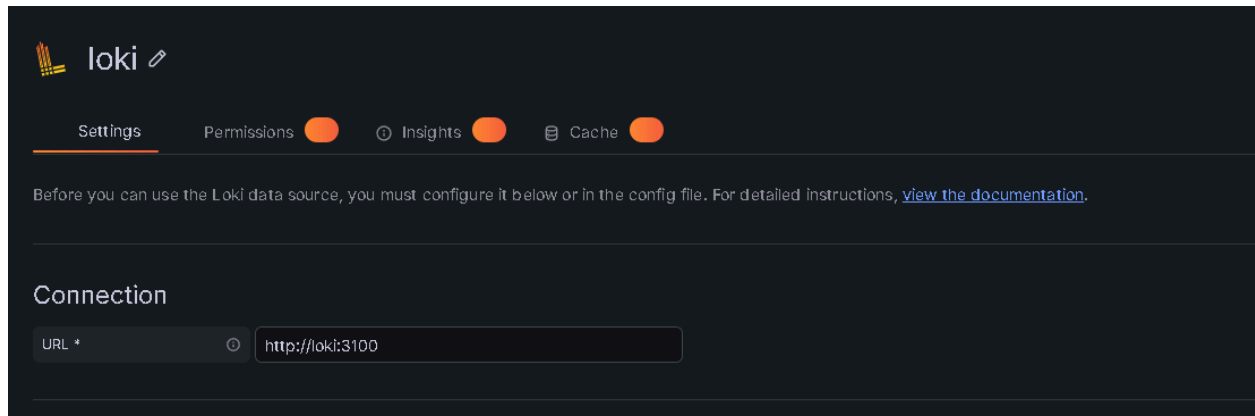
2. Archivos de la carpeta Logs



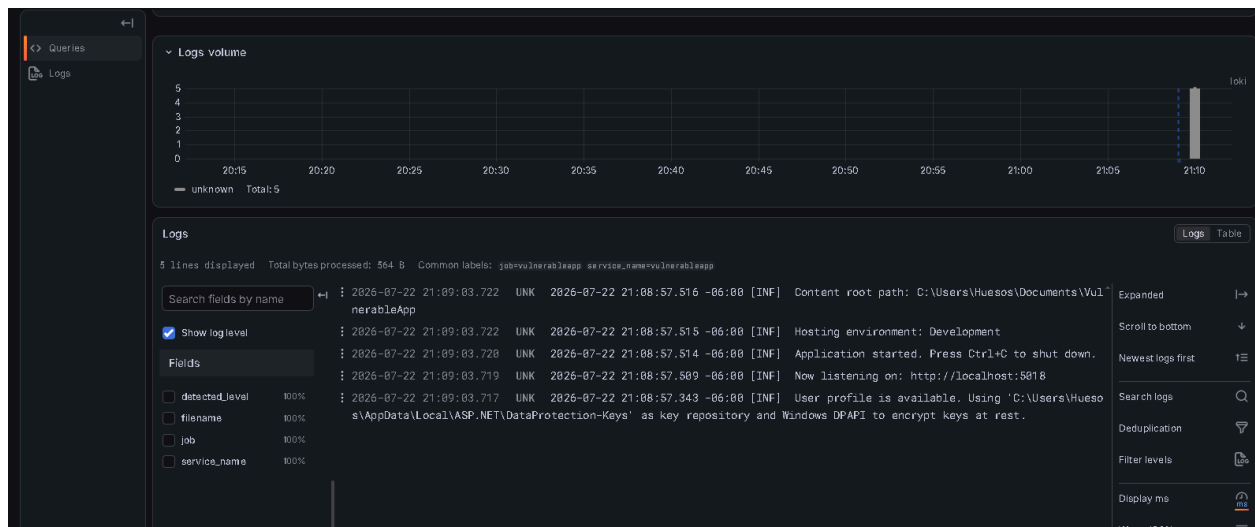
3. Seq mostrando los eventos

```
22 Jul 2026 21:08:57.516 Content root path: C:\Users\Huesos\Documents\VulnerableApp
22 Jul 2026 21:08:57.515 Hosting environment: Development
22 Jul 2026 21:08:57.514 Application started. Press Ctrl+C to shut down.
22 Jul 2026 21:08:57.509 Now listening on: http://localhost:5018
22 Jul 2026 21:08:57.343 User profile is available. Using 'C:\Users\Huesos\AppData\Local\ASP.NET\DataProtection-Keys' as key repository and Windows DPAPI to encrypt keys at rest.
16 Jul 2026 20:10:09.111 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 11.5707ms
16 Jul 2026 20:10:09.110 HTTP GET /Home/Error responded 200 in 7.9738 ms
16 Jul 2026 20:10:09.109 Executed endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.108 Executed action 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)' in 3.3255ms
16 Jul 2026 20:10:09.107 Executed ViewResult - view Error executed in 1.158ms.
16 Jul 2026 20:10:09.106 Executing ViewResult, running view Error.
16 Jul 2026 20:10:09.105 ● Error detectado. RequestId: 00-835a9ebb3ea2634b9fe8ef1c91d698aa-1f7b9d0268f7f88c-00
16 Jul 2026 20:10:09.103 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller 'VulnerableApp.Controllers.HomeController (...)'
16 Jul 2026 20:10:09.102 Executing endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.100 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
16 Jul 2026 20:10:09.065 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 11.0777ms
16 Jul 2026 20:10:09.064 HTTP GET /Home/Error responded 200 in 7.5767 ms
16 Jul 2026 20:10:09.063 Executed endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.062 Executed action 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)' in 3.0266ms
16 Jul 2026 20:10:09.061 Executed ViewResult - view Error executed in 1.0895ms.
16 Jul 2026 20:10:09.060 Executing ViewResult, running view Error.
16 Jul 2026 20:10:09.059 ● Error detectado. RequestId: 00-0e8d763b2a8a6a999335616c85161bbb-56d212aab642c838-00
16 Jul 2026 20:10:09.057 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller 'VulnerableApp.Controllers.HomeController (...)'
16 Jul 2026 20:10:09.056 Executing endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.054 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
16 Jul 2026 20:10:09.020 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 11.161ms
16 Jul 2026 20:10:09.018 HTTP GET /Home/Error responded 200 in 7.6349 ms
16 Jul 2026 20:10:09.017 Executed endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.016 Executed action 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)' in 3.1713ms
16 Jul 2026 20:10:09.015 Executed ViewResult - view Error executed in 1.0125ms.
16 Jul 2026 20:10:09.014 Executing ViewResult, running view Error.
16 Jul 2026 20:10:09.013 ● Error detectado. RequestId: 00-6a2368ea76f2d23caaaab43fb577b0d45-7fee47effda0b4a-00
16 Jul 2026 20:10:09.011 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller 'VulnerableApp.Controllers.HomeController (...)'
16 Jul 2026 20:10:09.011 Executing endpoint 'VulnerableApp.Controllers.HomeController.Error (VulnerableApp)'
16 Jul 2026 20:10:09.008 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
16 Jul 2026 20:10:08.973 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 11.6298ms
```

4. Grafana con Loki configurado



5. Consulta filtrada mediante labels



6. docker ps

Search															Only show running containers														
		Name	Container ID	Image	Port(s)		CPU (%)	Memory usage	Memory (%)	Disk read/write	Network I/O	PIDS		Last started	Actions														
		seq	86b39df6a886	datastus:53415341	Show all ports (2)		1.37%	154.1MB / 3.5GE	4.29%	320MB / 8.99MB	45.6KB / 107KB	67		4 minutes ago															
		vulnerableapp	-	-	-		4.91%	621.35MB / 10.5	17.33%	1.75GB / 72.3MB	18.62MB / 1.79v	21, 23, 14		4 minutes ago															
		loki	8719db47593a	grafana/loki	3100:3100		1.84%	156.3MB / 3.5GE	4.36%	373MB / 2.09MB	12.2KB / 17KB	21		4 minutes ago															
		grafana	d93c459deaca	grafana/grafana	3000:3000		1.72%	405.2MB / 3.5GE	11.3%	1.13GB / 68.5MB	18.6MB / 1.77ME	23		4 minutes ago															
		promtail	0925eb253370	grafana/promtail			1.35%	59.86MB / 3.5GE	1.67%	257MB / 1.71MB	3.56KB / 2.35KB	14		4 minutes ago															
		sonarqube-infr	-	-	-		9.79%	1.55GB / 7.01GB	44.23%	1.76GB / 953.67	667KB / 663KB	302, 26		5 minutes ago															
		sonarqube	87a42dcecdca	sonarqube/sonarqube	9000:9000		9.48%	1.48GB / 3.5GB	42.15%	1.65GB / 950MB	286KB / 379KB	302		5 minutes ago															
		sonar_db	2127e473e5a8	postgres:11			0.31%	74.59MB / 3.5GE	2.08%	112MB / 3.67MB	381KB / 284KB	26		5 minutes ago															

7. docker-compose.yml actualizado

```
1  version: "3.9"
2
3  services:
4    grafana:
5      image: grafana/grafana:latest
6      container_name: grafana
7      ports:
8        - "3000:3000"
9      volumes:
10       - grafana_data:/var/lib/grafana
11      networks:
12        - observabilidad_net
13
14    loki:
15      image: grafana/loki:latest
16      container_name: loki
17      ports:
18        - "3100:3100"
19      volumes:
20       - loki_data:/tmp/loki
21      networks:
22        - observabilidad_net
23
24    promtail:
25      image: grafana/promtail:latest
26      container_name: promtail
27      volumes:
28        - ./Logs:/var/log/VulnerableApp:ro
29        - ./promtail-config.yaml:/etc/promtail/config.yaml:ro
30      command: -config.file=/etc/promtail/config.yaml
31      networks:
32        - observabilidad_net
33
34  networks:
35    observabilidad_net:
36      driver: bridge
37
38  volumes:
39    grafana_data:
40    loki_data:
```


8. Archivo de configuración de Promtail

```
1  server:
2    ..http_listen_port: 9080
3    ..grpc_listen_port: 0
4
5  positions:
6    ..filename: /tmp/positions.yaml
7
8  clients:
9    ...url: http://loki:3100/loki/api/v1/push
10
11  scrape_configs:
12    ...job_name: vulnerable_app_logs
13    ...static_configs:
14      ...- targets:
15        ...- localhost
16      ...- labels:
17        ...- job: vulnerableapp
18        ...- __path__: /var/log/VulnerableApp/*.txt
19      ...- pipeline_stages:
20        ...- regex:
21          ...- expression: "(?P<categoria>(Security|Audit))"
22        ...- labels:
23          ...- categoria:
24
```

Conclusiones

Se logró extender la solución de logging de VulnerableApp con una plataforma de observabilidad basada en Grafana, Loki y Promtail sin modificar la aplicación ni su lógica de negocio, cumpliendo con la restricción de no contenerizarla. El bind mount de la carpeta Logs permitió que Promtail accediera a los mismos archivos de texto que ya generaba Serilog, y el pipeline de labels (job, categoria) sentó la base para diferenciar registros de VulnerableApp, Security y Audit dentro de Loki.

El análisis comparativo evidenció que Seq y Grafana+Loki no son herramientas competidoras sino complementarias: Seq resulta más adecuado para la depuración detallada de una sola aplicación .NET durante el desarrollo, mientras que Grafana+Loki aporta una capa de observabilidad centralizada, económica en almacenamiento (al indexar solo labels) y capaz de combinar logs con métricas y alertas en un entorno con múltiples servicios.

Como punto de seguimiento, se recomienda validar en la evidencia que el label categoria efectivamente se esté generando (dado que depende de que las líneas de log contengan literalmente las palabras 'Security' o 'Audit'), ajustando el regex o el

contenido de los mensajes si fuera necesario para que la consulta filtrada del Reto devuelva resultados.